

FlashDMoE: 单一内核中的快速分布式混合专家

Osayamen Jonathan
Aimuyo*康奈尔大学
oja7@cornell.edu

Byungsoo Oh康奈尔大学
bo239@cornell.edu

Rachee Singh康奈尔大学
rs2293@cornell.edu

摘要

专家混合 (MoE) 模型的计算稀疏性使得随着模型规模增大, 计算成本呈次线性增长, 从而为训练大规模神经网络提供了一条可扩展的路径。然而, 现有实现存在 GPU 利用率低、显著的延迟开销以及无法利用任务局部性等根本性问题, 主要源于 CPU 管理的调度、主机发起的通信以及频繁的内核启动。为克服这些局限性, 我们开发了 FlashDMoE, 这是一种完全 GPU 驻留的 MoE 算子, 将专家计算与 GPU 间通信融合进单一持久化 GPU 内核。FlashDMoE 使分发、计算和合并阶段实现细粒度流水化, 消除启动开销并减少空闲间隙。不同于现有工作, FlashDMoE 摒弃批同步集体通信, 改用单边、设备发起的 GPU 间 (R) DMA 传输, 从而解锁载荷效率, 在稀疏激活层中消除臃肿或冗余的网络载荷。在配备 8×H100 GPU 的节点上, 针对最多包含 128 名专家和 16K 标记序列的 MoE 模型进行评估时, FlashDMoE 相较于最先进的基线方法, 实现了 GPU 利用率最高提高 9×、延迟最高降低 6×、5.7× 更高的吞吐量以及 4× 更好的重叠效率——尽管我们使用 FP32, 而基线使用 FP16。FlashDMoE 表明, 原则性的 GPU 内核-硬件协同设计是解锁大规模分布式机器学习性能上限的关键。

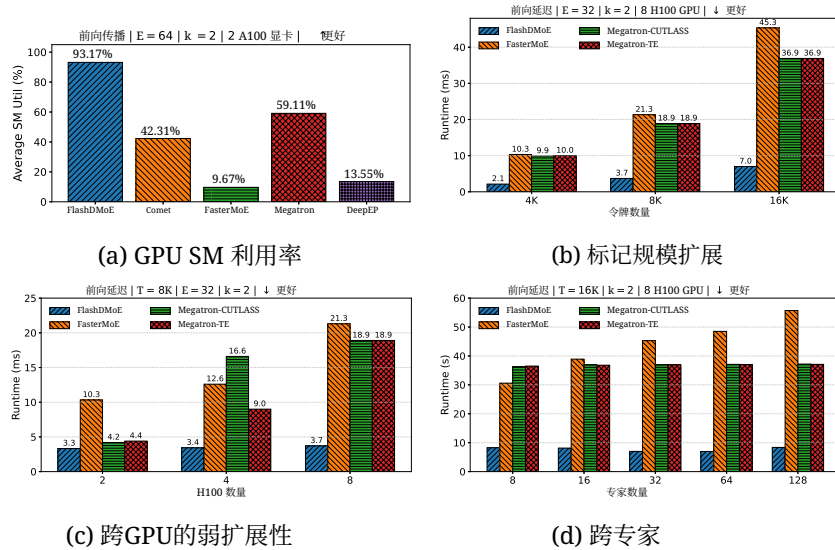
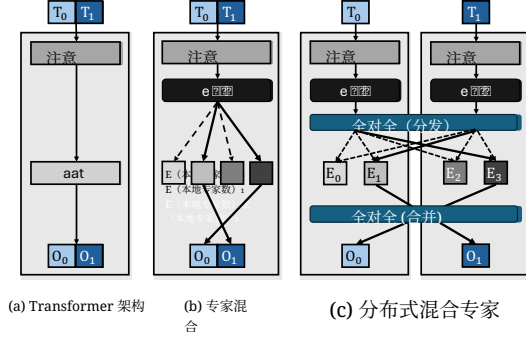


图1: FlashDMoE 性能。

*通信请发至 osayamen@stanford.edu

1 引言

包括 DeepSeek-v3 [1], LLama4 [2], DBRX [3] 和 Snowflake Arctic [4], 在内的最先进的大型语言模型 (LLMs) 因其计算效率和在众多任务上的强劲表现, 已采用专家混合 (MoE) 架构。传统的 Transformer 块由一个自注意力模块, 后接一个稠密前馈网络 (FFN) [5] 组成。相比之下, MoE 架构将这一单个 FFN 替换为多个规模相同的 FFN, 也称为专家 (图 2(b))。一个可训练的神经网络, 称为门控函数, 在运行时通过将输入标记动态路由到选定的专家来稀疏地激活这些专家。这种模型参数 (更多 FFN) 的增加在不伴随相应增加的计算成本的情况下提升了模型质量。T、E 和 O 分别表示输入标记、专家和输出激活。



MoE 中的通信开销。 随着专家混合模型规模的增长, GPU 内存限制使得无法在单个设备上容纳所有专家。标准做法是使用专家并行 (EP) 将专家分布到多个 GPU 上, 这需要通过在 *AllToAll* (全互连) 或 *AllGather* (全收集) [1, 4, 3, 6] 中的多对多通信来进行令牌路由。还需要再进行一轮上述多对多通信, 以将专家处理过的被置换的标记恢复为序列中的原始顺序。现有工作观察到这些通信操作占总运行时间的 68% [7, 8], , 在此期间 GPU 完全空闲, 除非实现显式与计算重叠。这种流水化难以高效实现, 因为它需要异步 GPU 驱动的通信和内核融合以最大化重叠效率。通常, PyTorch 等框架提供的 GPU 间通信 API 并非此类, 而是 CPU 驱动 [9]。

MoE 中的内核启动开销。 通信重叠的效果还进一步受到从 CPU 启动大量内核所带来的开销的限制。具体而言, 现有实现 [10–13] 每执行一次单层就需要启动大量内核 (见表1)。频繁的内核启动会通过以下方式对性能产生负面影响: (1) 在 GPU 之间造成非确定性的内核启动时间, 加剧慢节点问题; (2) 引入不必要的同步点, 导致 GPU 在继续之前需要等待其他 GPU 或 CPU; (3) 在内核边界产生重复的全局内存往返。尽管在静态工作负载中, CUDA 图 [14] 可以在一定程度上缓解第一个问题, 但它与专家混合的动态专家路由模式不兼容。解决其余问题需要新的方案, 我们在本文中通过完整的内核融合和异步设备发起的通信来实现。

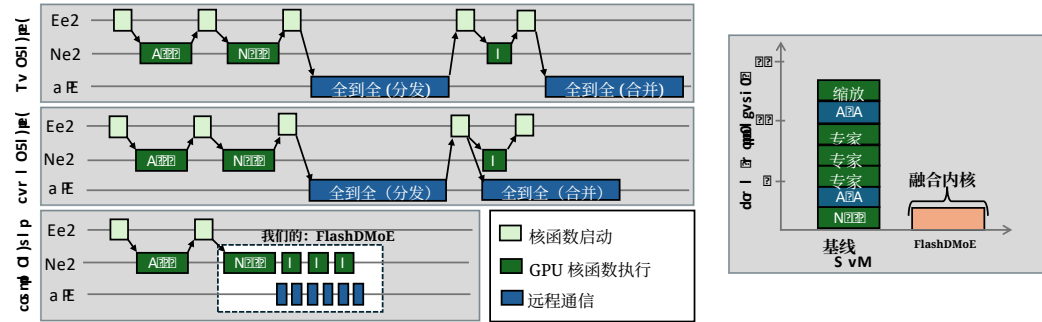


图3: 比较 FlashDMoE 与最先进技术: 这些技术要么不对通信与计算进行重叠 (左上), 要么进行一定程度的重叠 (左中)。FlashDMoE 是一个常驻内核, 将 MoE 算子的所有计算与通信进行融合 (左下)。FlashDMoE 实现了设备发起的计算 (门控、专家前馈网络 (FFN)、缩放) 以及通信任务 (右)。

表1: 内核融合比较。我们的方法是首个将 DMoE 层完全融合到单一 GPU 内核中的方法。我们基于 Nsight Systems 的性能分析报告 GPU 操作。我们在 2 张 A100 上统计单个层（门控 → 分发 → 专家 → 合并），每张 GPU 有 32 个专家。

工作	启动的 GPU 操作
FlashDMoE	1
COMET [11]	33
Megatron-LM CUTLASS [12, 15]	85
Megatron-LM TE [12, 15]	261
Megatron-LM + DeepEP [1]	432
DeepSpeedMoE [10]	550

1.1 我们的贡献：单个内核中的 DMoE

为克服最先进的 MoE 模型中的这些根本性低效，我们开发了 FlashD- MoE，将所有 DMoE 计算与通信任务融合到一个单一持久化 GPU 内核中 即，一个在整个 MoE 算子期间保持活跃的内核（图3 左下）。与其由 CPU 协调多次内核启动，FlashDMoE 只需启动一个内核，从而显著减少 CPU 的参与。在该融合内核内，FlashDMoE 采用响应式编程模型，在数以万计的 GPU 线程之间实现细粒度并行以及松耦合的非阻塞执行。

内核内块调度和瓦片并行。 FlashDMoE 实现了瓦片级并行性，这意味着它将输入标记矩阵划分为更小且相互独立的单元，称为瓦片，这些单元由线程块处理，但由 warp（束）进行管理（调度和构建）。我们将除一个之外的每个线程块专门化为处理器 以执行计算。此外，我们指定一个专用的操作系统（OS）块（4 个 warp）来执行管理任务：(1) 将计算工作调度到处理器（调度器），以及 (2) 从其他 GPU 接收的消息中解码计算任务（订阅者）。这种设计允许 FlashDMoE 基于就绪度动态地将任务分配给 GPU 线程块，确保在 DMoE 算子整个生命周期内，没有任何 GPU SM（流式多处理器）处于空闲状态。FlashDMoE 选择 tile 尺寸以最大化 GPU 的算术强度，同时仍能从高度的并行性中受益。

异步且负载高效的通信。 通过自底向上重新设计 MoE 算子，FlashDMoE 解决了传统 MoE 执行流水线中固有的根本性低效问题。一个显著的低效点是通信过程中的 token 填充。为简化编程复杂度且由于集合通信 API 的对称性约束，现有实现必须对 token 载荷进行零填充以匹配预定义的缓冲区大小。当 Token 被非对称路由到专家时就会发生这种情况，导致 GPU 的实际接收量远低于预期容量。然而，这些空载荷会浪费通信带宽，增加数据传输时延，并且在某些实现中可能导致在空矩阵上进行不必要的计算。FlashDMoE 通过仅将未填充的 token 发送到具有被选中的专家的 GPU，引入负载高效的 通信，从而节省通信和计算资源。

技术挑战。 要实现 FlashDMoE 的单内核设计并达到高性能，需要解决若干技术挑战：

(1) 轻量级计算依赖管理；(2) 探索最优 SM 占用配置；(3) 实现设备内 BLAS 操作；(4) 最小化设备间与设备内同步开销；(5) 在可用时利用基于统一虚拟寻址（UVA）的 DMA 实现传输感知。在解决这些挑战的过程中，FlashDMoE 的设计与传统的同步 *AllToAll*（全互连）集体通信有着根本性的不同，后者在层执行期间 GPU 存在显著的空闲时间。对于设备发起的通信，FlashDMoE 使用 NVSHMEM [16] 来建立跨所有 GPU 的全局地址空间，以实现前述的直接内存访问（DMA）或远程直接内存访问（RDMA）通信。对于设备内 BLAS，FlashDMoE 通过 CUTLASS [17] 开发了定制的高性能 GEMM 运算。

结果。 我们在分布于多个节点的多个 GPU 上对 FlashDMoE 进行评估。我们的评估显示，与最先进实现相比，FlashDMoE 实现了 6× 的时延加速比、9× 更高的 GPU 利用率、4× 更好的弱扩展效率，以及 5.7× 更高的吞吐量。我们

我们预计这些性能提升在多节点场景下会更好，其中节点间通信通过低带宽的节点间链路进行(例如，RDMA（远程直接内存访问），InfiniBand)。

2 动机

2.1 同步通信与掉队者

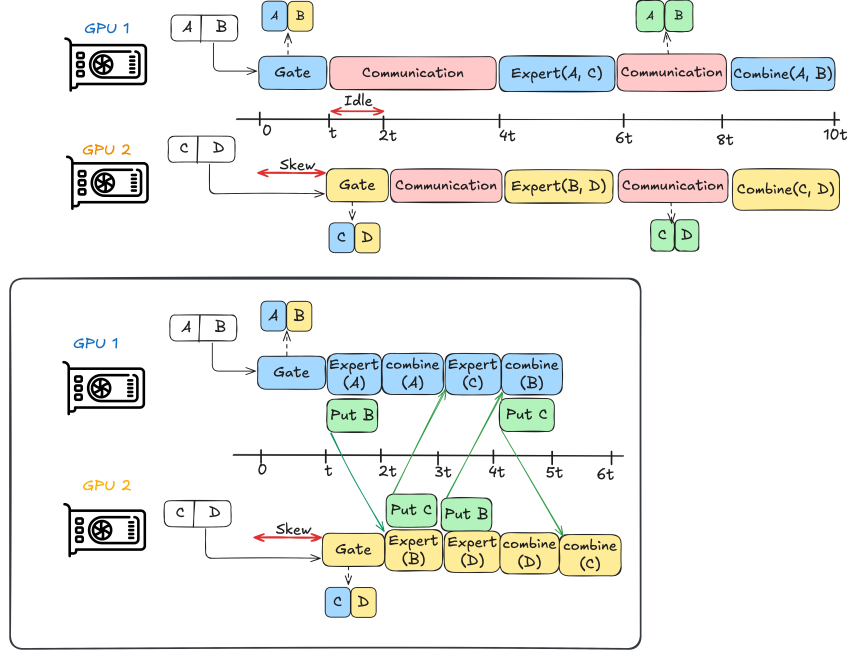


图4：重叠调度（下）展示了如何将顺序调度（上）中的空闲时间重新用于计算。FlashDMoE 实现了重叠调度。

AlltoAll 或 *AllGather* 通信（目前在 MoE 框架中使用）是一种 *synchronous* 集合操作，其完成需要所有相关 GPU 的参与。在这种情况下，GPU 之间处理速度或内核调度的差异会引发不利的掉队者效应（图 4），从而损害（1）该集合操作的性能和（2）端到端性能，因为在集合操作结束之前，停滞的 GPU 无法继续进行下游的依赖或独立任务。具体而言，如图15所示，对于在 32 张 A100 GPU 上分布式训练一个13亿参数的 GPT-3 MoE 模型，与图15b中实际内核时间的平均值相比，我们观察到 P95（第95百分位）通信性能下降**1.32X**。这种性能下降相对温和，因为底层硬件是一台针对“软件抖动”[18]进行了良好调优的超级计算机。然而，在单节点虚拟机（VM）中，我们观察到更为严重的 P95（第95百分位）性能损失**11X**。与先前的高性能计算工作一致 [19,20]，我们认为，去除这种同步集合通信中固有的屏障，将允许 GPU 如图4所示将观察到的空闲时间重新用于下游计算。

表2：同步全对全通信中的掉队者延迟。我们捕获了多个步骤中由掉队者引起的延迟分布。令**实际时间** t_a 表示所有 GPU 中最快的内核执行时间，**总时间** t 为最大记录的 step 时间。我们将 *Delay* 定义为 t 与 t_a 之间的最大差值。注意，*Delay* 是空闲时间。对于 1×8 V100，我们分析了 1750 步；对于 8x4 A100，我们分析了 600 步。原始分布见图15。

系统	# 节点	# GPU	中位数	p95
商用 VM (V100)	1	8	3.1x	11.4 倍
超级计算机 (A100)	8	32	1.09 倍	1.32x

2.2 内核启动开销。

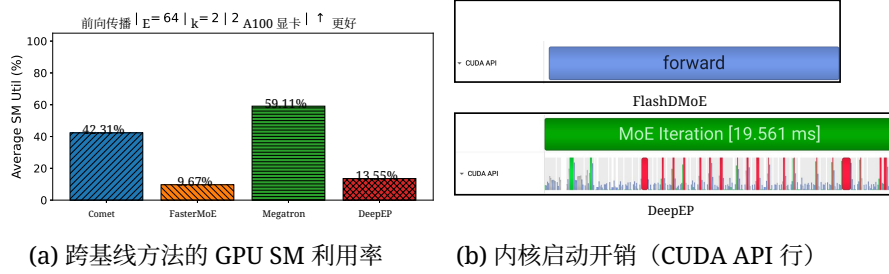


图5: 图 5a 展示了在 2 个 NVLink 互连的 A100 (单向带宽 300 GB/s) 上进行 100 次专家混合前向传播时的平均 GPU 利用率。尽管互连带宽很高, 我们仍观察到高达 90% 的空闲时间, 我们将其归因于内核启动间隙和非重叠通信。

我们比较了 FlashDMoE 与现有基线方法的内核启动开销。表1显示了单次前向传播期间的内核启动次数: FlashDMoE 精确地启动一个常驻内核, 而基线方法为完成相同的计算会启动多达 550 个短时空核。图5基于 Nsight Systems 捕获的 CUDA API 跟踪提供了可视化对比, 展示了 FlashDMoE 与 DeepEP 的差异。DeepEP 呈现出大量细碎的 CUDA API 调用, 且在各个算子之间频繁停顿, 导致 GPU 空闲时间增加 (图5a)。相比之下, FlashDMoE 通过避免启动开销和同步间隙来维持较高的 GPU 利用率——实现了93.17%的 GPU 利用率, 而 DeepEP 仅为 14%。实验结果见第4节, 相关工作的讨论见附录A。

3 融合式 MoE 内核设计

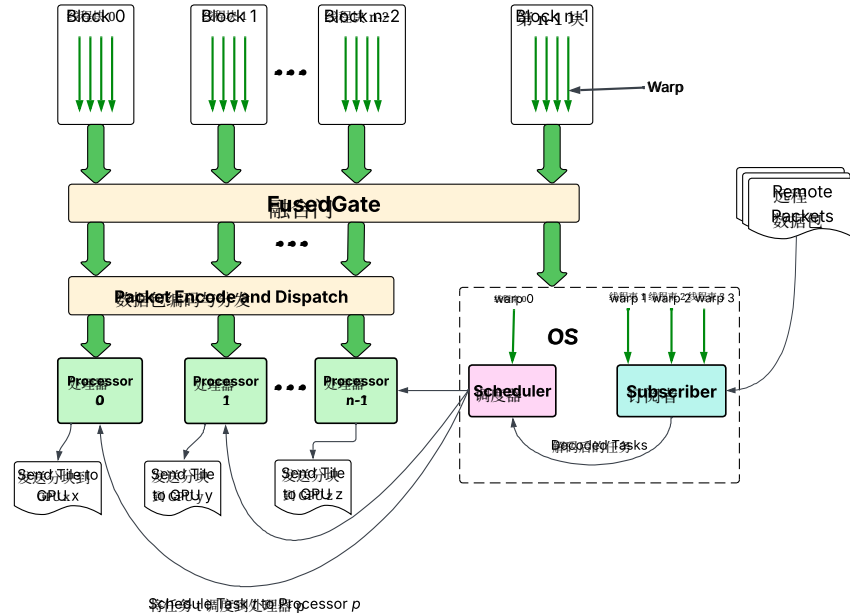


图 6: FlashDMoE 融合内核

现代分布式 MoE 系统存在两个局限性: (1) 在关键路径上频繁发生多对多 (*AllToAll* (全互连) 或 *AllGather* (全收集)) 集体通信; (2) 重复的内核启动带来显著的开销。我们在 FlashDMoE 中解决了这些问题, 它是以单一持久化 GPU 内核实现的完全融合的 MoE 算子。不同于以往的方法 [11, 1, 10, 12, 21, 8, 22–26], FlashDMoE 首次实现了完全融合的分布式 MoE 内核, 只需单次内核启动即可完全消除内核启动开销 (见 表1)。

算法 1: FlashDMoE 分布式混合专家 融合内核

输入: $A, O \in \mathbb{R}^{S \times H}$, $X \in \mathbb{R}^{E \times H \times D}$, N

```

1 开始
2    $T_\phi, G_\phi \leftarrow \text{融合门}(A)$ 
3   如果 块ID + 1 <  $N$  则
4       分发( $T_\phi, A$ )
5       处理器::启动()
6   else
7       if warpID == 0 then
8           调度器::启动()
9       else
10          订阅者::启动( $T_\phi, G_\phi, O, X$ )
11      结束 if
12 结束 if
13 结束

```

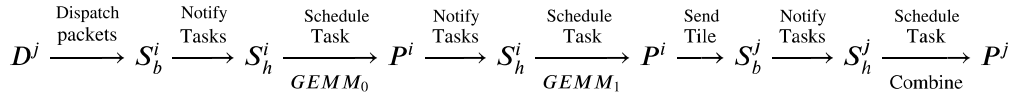


图 7: DMoE 功能依赖表示为一条 Actor 交互链。我们将 S_b 、 S_h 和 P 分别记为订阅者、调度器和处理器参与者。对于任意 Actor $a \in \{S_b, S_h, P\}$, a^i 标识 GPU i 上的一个 Actor。我们将 D_i^j 定义为算子, 其中 GPU j 向 GPU i 分发瓦片的数据包。该图在瓦片粒度上表达了任务依赖, 即 $GEMM_0$ 、 $GEMM_1$ 、合并与通信共同产生一个输出瓦片。通知以信号的形式传播: 通过共享内存 (订阅者 $\leftrightarrow$ 调度器) 或全局内存 (调度器 $\leftrightarrow$ 处理器, 或 GPU 间通信)。注意, 单边 GPU 间传输 (数据包或单个瓦片) 会被耦合到一个信号, 以通知接收方 GPU j 上的 S_b^j 消息已送达。

基于 Actor 的模型。 FlashDMoE 的设计基于并发计算的 Actor 模型 [27–29]。我们通过将 GPU 线程块和 warp (束) 专用为三种不同的 Actor 角色来实现该模型: (1) **处理器** (§E.1), (2) **订阅者** (§E.3), 以及 (3) **调度器** (§E.2)。处理器执行计算 (GEMM 运算和逐元素运算) 以及瓦片通信。我们使用 CUTLASS [17] 作为高性能 BLAS 例程的底层基础设施, 并使用 NVSHMEM 进行内核发起的通信 [16]。订阅者和调度器承担管理职能。具体而言, 调度器将计算任务分配给可用的线程块。我们的关键创新是使调度器同时具备多线程的能力, 以实现高调度吞吐量, 并且工作守恒, 从而确保 GPU SM 利用率始终保持较高。另一方面, 订阅者将来自对等 GPU 的瓦片数据包解码为任务描述符 (§3.1)。在一个 GPU 上的 N 个线程块中, 我们将 $N - 1$ 个专用于承担 **处理器** 角色。我们将最后一个线程块专用于操作系统 (OS)。在该线程块内, 我们使三个 warp (束) 承担 **订阅者** 角色, 一个 warp (束) 承担 **调度器** 角色。这种在各个 Actor 之间划分线程块的做法是有意为之: 我们的目标是在管理任务上使用尽可能少的资源, 同时将大部分资源保留用于执行专家混合计算任务。图 6 总结了 FlashDMoE 的体系结构及其组成的 Actor, 而算法 1 则以代码对该系统进行了非常贴近的还原。请注意, $A \in \mathbb{R}^{S \times H}$ 是输入令牌矩阵; $O \in \mathbb{R}^{S \times H}$ 是输出矩阵; $X \in \mathbb{R}^{E \times H \times D}$ 是一个由专家权重构成的三维张量, 其中 E 表示执行 GPU 的本地专家数, H 是嵌入维度, D 是 FFN 中间维度, S 是序列长度。 $T_\phi \in (\mathbb{R}^2)^{E \times C}$ 是一个路由表数据结构, 其中 $T_\phi(e, c) = (i, w)$ 表示标记 i 在槽位 c 分发到专家 e 。 w 是组合权重 (方程 2), C 是专家容量。 T_ϕ 的元组结构属于实现细节。 $G_\phi \in \mathbb{R}^{S \times E}$ 记录由门控产生的亲和度分数 (方程 3)。

FlashDMoE 中的参与者间交互。 FlashDMoE 以瓦片的粒度 (张量的固定大小的分区) 对 MoE 计算和通信进行分解, 从而实现并行执行和任务的高效重叠。每个瓦片对应一个由任务描述符封装的离散工作单元。 **订阅者** 会从其接收的远程瓦片数据包中解码这些任务描述符。同时, **调度器** 接收关于可用任务的通知, 并将其分发给 **处理器 Actor** 来执行这些任务所定义的计算, 即

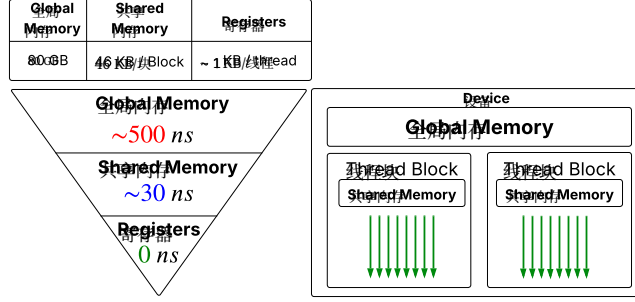


图8: GPU内存层次结构。左侧的倒金字塔展示了加载/存储访问延迟 [30–32]。上方的表格概述了不同内存层级的容量（针对 A100 GPU）。共享内存和寄存器容量是 FlashDMoE 的静态配置。右图展示了可访问范围：片上寄存器的范围限定为一个线程；片上共享内存 对一个线程块内的所有线程可见；片外全局内存 可被设备上的所有线程访问。

前馈网络（FFN）和专家合并操作。图 7 展示了 Actor 交互链，说明了 FlashDMoE 如何强制执行 DMoE 的功能依赖。

在 FlashDMoE 中确定tile尺寸。 在 FlashDMoE 中选择合适的tile尺寸对于确保高效的GPU利用率至关重要。过小的瓦片会导致GPU 未被充分利用，而过大的瓦片会产生寄存器压力，导致寄存器溢出到本地内存，从而降低性能。经过仔细的参数扫描，我们选择了tile尺寸为 (128, 64)。我们的关键见解是：增大tile宽度会显著提升每线程寄存器使用量，可能触发代价高昂的溢出；在不调整线程数的情况下增大tile高度会增加每线程工作量，损害性能。将每块线程数提高到超过我们固定的 128 个线程会减少并发块数量，进而对SM占用率产生负面影响。更大的线程块尺寸还会增加块内同步（线程块同步屏障）的开销，进一步降低性能。因此，我们选择的tile尺寸在寄存器使用、共享内存约束和GPU占用率之间实现了平衡，从而获得最佳性能。

3.1 用于计算的任务抽象

计算算子。 FFN算子是一种在 Transformer架构 中广泛使用的标准逐位置前馈网络 [5]，由两个线性变换构成，中间通过一次非线性激活 ϕ (例如，GELU 或 ReLU)：

$$\text{FFN}(x) = W_2 \cdot \phi(xW_1 + b_1) + b_2 \quad (1)$$

在此， W_1 和 W_2 表示可学习权重矩阵，而 b_1 和 b_2 是偏置。专家合并操作用于诸如 GShard [33]和 DeepSeek [1], 等架构中，通过根据其亲和度分数计算加权组合来合并多个专家的输出：

$$C_i = \sum_{j=1}^k g_{i,e} \quad (2)$$

$$\mathbf{h}_i = \sum_{j=1}^k \frac{g_{i,e}}{C_i} \cdot \mathbf{h}_i^k \quad (3)$$

在这些方程中， $i \in 0, S-1$ 表示一个输入标记索引， $e = E_{i,k}$ 标识为标记 i 所选择的第 k 个专家，而 $g_{i,e}$ 是亲和度分数，用于指示专家 e 与标记 i 的相关性。

统一任务抽象。 我们将前馈网络和合并操作统一到一个称为任务的通用抽象之下。任务为订阅者、调度器和处理器之间的瓦片级工作提供统一的通信接口。形式化地，任务描述符 $t \in \mathcal{T}$ 被定义为一个元组：

$$t = (\mathcal{M}, \star, \phi)$$

其中, $\mathcal{M}$ 是一组元数据 (例如, 设备ID、块索引), $\star$ 是二元张量操作 (具体为, 矩阵乘法 $\cdot$ 或哈达玛积 $\odot$), 并且 ϕ 是逐元素激活函数 (例如, ReLU 或恒等函数)。

我们将一个任务 t 定义为在输入张量 A 、 B 、 D 上运行, 产生输出张量 C , 如下所示:

$$\mathcal{F}_t(A, B, C, D) := C \leftarrow \phi(A \star_t B + D) \quad (4)$$

算子 $\star_t$ (由 $\star$ 实例化) 可能会根据任务元数据 $\mathcal{M}$ 而表现不同, 且 $A \star_t B$ 的结果会累加到 D 中。我们在 §D 中提供了任务元数据的示例。

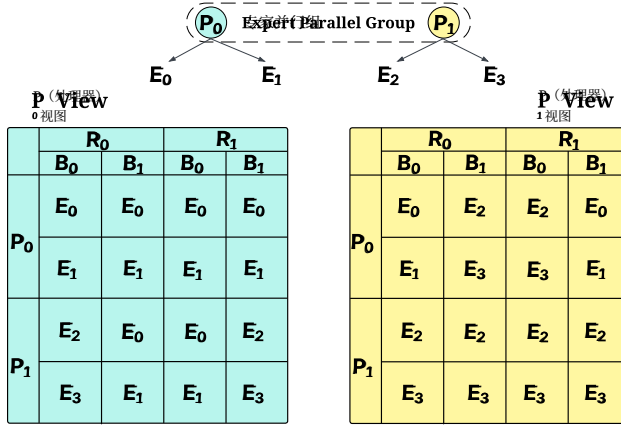
在实践中, 我们将公式(4)定义的每个任务实现为一个单一融合的 `--device--` 修饰的函数, 处理器 (算法2) 在运行时调用它。对 t 的融合包括将 ϕ 以及随后的加法操作应用于存储二元算子 $\star_t$ 结果的寄存器。为展示其灵活性, 我们给出如何使用该任务框架来表述前馈网络和专家合并操作。注意, 为简化起见, 我们省略了矩阵乘法符号 ($\cdot$)。此外, ϕ_1 可以是任意激活函数, 而 ϕ_2 是恒等函数。前馈网络表达为:

$$\begin{aligned} t_1 &= (\mathcal{M}, \cdot, \phi_1), \quad t_2 = (\mathcal{M}, \cdot, \phi_2), \\ \mathcal{F}_{t_1}(A, B_1, C_1, D_1) &:= C_1 \leftarrow \phi_1(AB_1 + D_1), \\ \mathcal{F}_{t_2}(C_1, B_2, C_2, D_2) &:= C_2 \leftarrow \phi_2(C_1B_2 + D_2). \end{aligned}$$

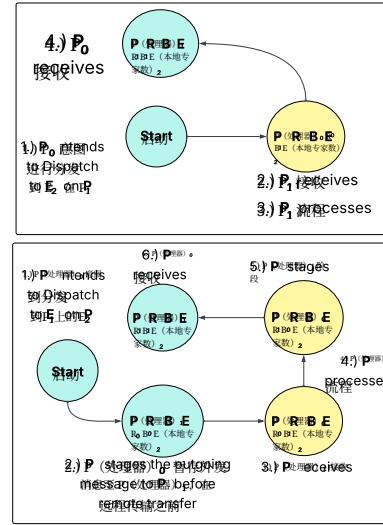
而专家合并操作形式化为:

$$\begin{aligned} t_3 &= (\mathcal{M}, \odot, \phi_2), \\ \mathcal{F}_{t_3}(A, S, C, C) &:= C \leftarrow \phi_2(A \odot S + C). \end{aligned}$$

3.2 用于跨GPU通信的对称张量布局



(a) 跨 2 个专家并行进程的对称张量布局。



(b) 用于 DMA (直接内存访问) (上) 和 RDMA (远程直接内存访问) (下) 通信的状态机。

在单张GPU设备内, FlashDMoE 中的 Actor 通过 GPU 的内存子系统进行通信 (见图8)。具体而言, 调度器和订阅者执行体通过高速共享内存交换数据, 而其他 Actor 对通过全局内存通信。对于跨多个设备的通信, FlashDMoE 使用设备发起的通信, 利用单边 PGAS (分区全局地址空间) 编程模型 [34]。然而, 在 PGAS 中在不引入高成本同步的情况下实现可扩展且正确的单边内存访问是一项众所周知的挑战 [1, 35]。我们通过一种可被证明正确且可扩展的方案来应对这一挑战: 一种对称张量布局 L , 支持完全非阻塞内存访问。我们将 L 定义为:

$$L \in \mathbb{R}^{P \times R \times B \times E \times C \times H}$$

其中： P 是专家并行世界规模， R 标识通信轮次（即，两轮，一轮用于令牌分发，一轮用于合并）， B 是暂存缓冲区数量， E 是本地专家数， C 是扩展的专家容量 (§3.2.1)， H 是令牌嵌入维度。我们实现非阻塞通信的核心洞见是时间缓冲。具体而言，我们为底层令牌矩阵超额预留内存至少 $2 \cdot r$ 倍，其中 r 是依赖图中的通信轮次，且系数 2 用于为每个通信轮次中的入站和出站数据提供独立缓冲区。对于 MoE 模型，我们有 $2 \cdot r = 4$ 。这种适度的内存使用增加消除了单边数据传输期间对同步的需求。图9b 展示了在该对称张量布局中如何为单元建立索引，并将其用于直接内存访问（DMA）和远程直接内存访问（RDMA）操作。正如定理3.1 所强调的，这一基于 L 的索引方案是实现完全非阻塞访问并省去同步的底层机制，因为所有访问都是写操作无冲突。证明见第C节。

Theorem 3.1. 对称张量布局 L 是写写无冲突的。

为了构建 L ，我们从原始令牌缓冲区 $T \in \mathbb{R}^{S \times H}$ 开始，其中 S 是序列长度， H 是令牌嵌入维度。我们首先将序列维度 S 重组为三个子维度，分别表示专家容量（ C ）、本地专家槽位（ E ）和专家并行世界规模（ W ），使得：

$$C \cdot E \cdot W = C \cdot E' = S', \quad \text{where } S' \geq S \text{ and } E' \geq E_W$$

在均匀专家分布的典型情况下（如图9a所示），我们有 $S' = S$ 和 $E' = E_W$ ，其中 E_W 是模型中的专家总数。因此，令牌缓冲区的大小为 $\text{Size}(T) = S' \cdot H$ 。在图9a中，标记为 E_i （具有 $i \in \{0, \dots, 3\}$ ）的每个单元是大小为 (C, H) 的矩阵。在扩展先前工作 [33, 11] 的基础上，我们引入了额外的时间维度 R （通信轮次）和 B （暂存缓冲区）。每个通信轮次都有两个固定的暂存槽位：一个用于出站令牌，另一个用于进站令牌。每个槽位按维度 P 进行索引，形成形状为 (S', H) 的张量。因此，张量大小 $\text{Size}(L)$ 通常至少是原始令牌缓冲区大小的四倍；在均匀专家分布的情况下将恰好大四倍。经验上，我们发现：

$$\text{Size}(L) \approx 4 \cdot \text{Size}(T)$$

3.2.1 就地填充以提升有效载荷效率

由于在 MoE 调度中 Token 的分布具有动态性且不均衡，[36]，GPU 通常接收到的 Token 少于其预定义的专家容量。当前的 MoE 框架 [10] 通常在计算前使用空令牌对这些缓冲区进行填充，导致通信载荷不必要地增加并降低性能。相比之下，我们提出就地填充，在本地对称张量缓冲区内直接执行填充，从而消除多余的网络通信。如图9a所示作为参考，每个单元格 E_i 的大小依据专家容量 C 确定。我们进一步将该容量对齐，以确保可被瓦片块大小 $bM = 128$ 整除，从而保证处理器线程在消耗远程 token 时能够进行安全且对齐的内存读取。这种就地填充策略会略微增加 L 的内存占用，具体如下：

$$\text{Size}(L) \approx \begin{cases} 4 \cdot \text{Size}(T), & \frac{S}{E} \geq bM \\ 4 \cdot \frac{bM \cdot E}{S} \cdot \text{Size}(T), & \text{otherwise} \end{cases}$$

4 实验

我们实现（第G节）并评估 FlashDMoE，并围绕五个指标进行评估：前向延迟（第4.2节）、GPU 利用率（第4.3节）、重叠效率（第4.4节）、吞吐量（第4.5节），以及专家可扩展性（第4.6节）。我们在一台配备 8 块 NVIDIA H100 80G GPU（通过 NVLink 互连）、125 GB 内存和 20 个 vCPU 的服务器上运行实验。我们使用 PyTorch 2.6.0、CUDA 12.8 和 Ubuntu 22.04。所有实验均使用配置为 16 个注意力头、嵌入维度为 2048、FFN 中间层大小为 2048 的 MoE Transformer 模型。所有实验均采用分布式数据并行（DDP）和专家并行。我们仅在单个 MoE 层上执行前向传播，并在 32 次预热轮次后测量 32 次前向传播的平均运行时间。我们使用容量系数为 1.0 的 top-2 路由。我们将 FlashDMoE 与若干最先进的 MoE 系统进行对比：

系统：(1) **Comet** [11], (2) **FasterMoE** [13], (3) **Megatron-CUTLASS** [37], 以及 (4) **Megatron-TE**: Megatron-LM 搭配 Transformer 引擎 [38]。Comet 依赖 `cudaMemcpyPeerAsync` [39], 而 FasterMoE 和 Megatron-LM 则仅使用 NCCL 进行通信。我们还在多节点环境中评估了 FlashDMoE, 并在第F节中讨论我们的发现。

4.1 期望特性

在我们的实验中, 我们观察到 Comet 在 8 块 GPU 上呈现出异常糟糕的性能数值, 因此, 我们不在 8 块 GPU 的评测中纳入其结果。注意我们使用 FP32 精度评估 FlashDMoE, 而所有基线方法使用 FP16。这样做的原因是: (1) 没有任何基线支持 FP32, (2) 受时间限制, 我们无法将系统在 FP16 下调优至峰值性能。更重要的是, 这种精度差异使 FlashDMoE 处于劣势, 因为它使通信与计算的精度加倍, 使我们的结果成为保守的下界。尽管如此, 正如下文所示, FlashDMoE 仍优于所有基线方法。

4.2 前向延迟

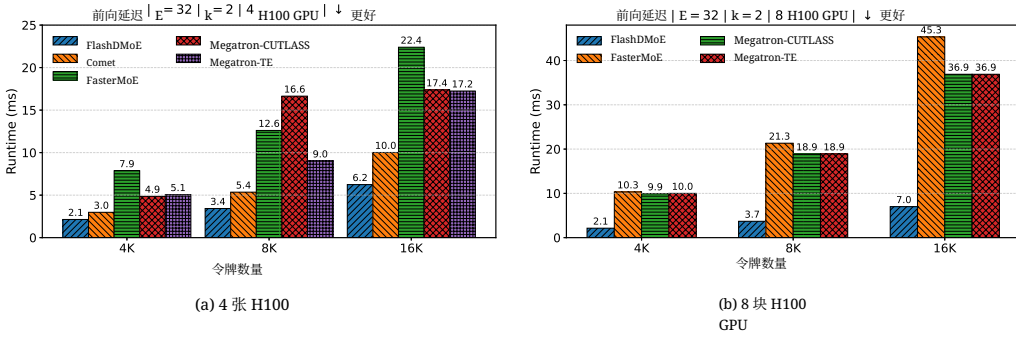


图10: 每块GPU的Number of Tokens 增加时的前向延迟。

我们首先在 4 块 GPU 和 8 块 GPU 的配置上 (图10), 测量 FlashDMoE 在不同序列长度下的前向延迟。FlashDMoE 持续优于所有基线方法, 且在更长的序列长度下改进尤为显著。在 4 块 GPU 上, 于 16K 个标记时相较 Megatron-TE 可获得最高 4.6 倍加速, 相较 FasterMoE 可达 2.6 倍。在 8 块 GPU 上优势更加明显, FlashDMoE 仍能保持低延迟; 而随着标记缓冲区按比例增大导致通信成本增加、基线方法性能急剧下降的情况下, FlashDMoE 相比之下可实现最高 6.4 倍加速。这些结果表明, FlashDMoE 能够在不遭受困扰其他实现的通信开销的情况下扩展标记吞吐量。

4.3 GPU 利用率

为量化 GPU 效率, 我们在前向传播期间测量流式多处理器 (SM) 利用率 (图11)。FlashDMoE 实现了 93.17% 的平均 SM 利用率, 比 FasterMoE (9.67%) 高出超过 9 倍, 比 DeepEP+ Megatron-LM (13.55%) 高出 6.8 倍, 比 Megatron-TE (59.11%) 高出 4 倍, 并且比 Comet (42.31%) 高出 2.2 倍。这一改进得益于我们的全融合内核架构以及对计算与通信任务的细粒度流水线。通过消除由内核启动导致的空闲间隙并启用内核内任务调度, FlashDMoE 确保在整个执行过程中 SM 始终忙于有效工作。

4.4 重叠效率

我们通过在 GPU 数量增加时 (图12b) 测量弱扩展效率, 来评估 FlashDMoE 在通信与计算之间的重叠程度。我们注意到, 大多数基线方法无法在单张 GPU 上运行, 因此我们使用 2 块 GPU 作为参照点。我们观察到 Megatron-CUTLASS 和 Megatron-TE 明显退化, 在 ≥ 4 块 GPU 时重叠效率下降到 0.5 以下。在 4 块 GPU 和 8 块 GPU 上, FlashDMoE 的效率分别最高可达 3.88x 和 4x。图12a 进一步说明了这一效率, 因为 FlashDMoE 展现出稳定的前向延迟增长,

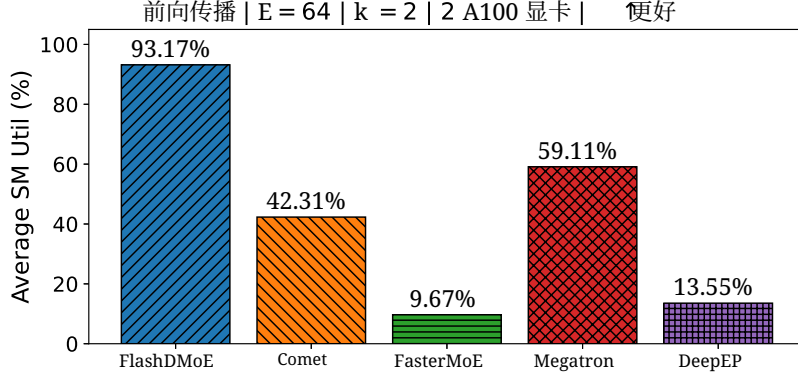
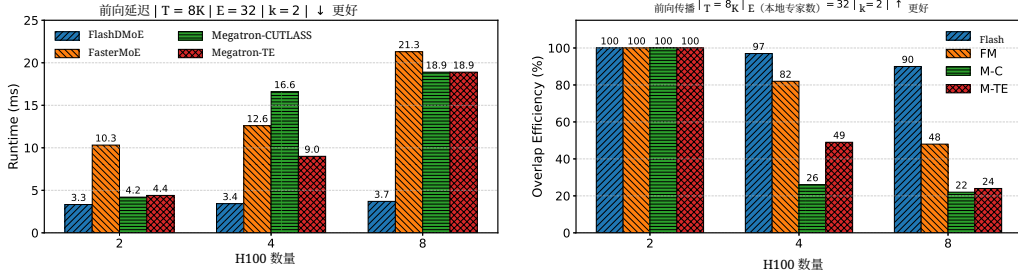


图11: SM 利用率的比较, 其定义为 SM 至少有一个线程束在执行的周期数与总周期数之比 [40]。数值表示 100 次迭代的平均 SM 利用率。所有实验使用 $T = 8K$ 和 E (本地专家数) = 64, 在两块 A100 显卡上进行



(a) 随着 GPU 数量增加的延迟。

(b) 弱扩展效率。

图12: 前向延迟随令牌数量在每个 GPU 上增加而变化。我们将重叠效率 O_e 定义为 $O_e = T(2)/T(N_G)$, 其中 $T(N_G)$ 表示在 N_G 个 GPU 上的延迟, $T(2)$ 表示在 2 块 GPU 上的延迟。

而基线方法 Megatron-CUTLASS 和 Megatron-TE 则经历了近似的延迟线性放大, 而 FasterMoE 表现为次线性扩展。我们将这种次优表现归因于慢节点效应和暴露的通信。相比之下, 在该弱扩展实验中, 由于每个 GPU 的工作负载固定, FlashDMoE 如预期表现出恒定延迟。这些结果进一步证实, FlashDMoE 的基于 Actor 的设计和异步数据移动实现了近乎理想的重叠, 即使在缩放时亦如此。

4.5 吞吐量

吞吐量, 以每秒 token 数 (MTokens/s) 计量, 反映了端到端系统效率。如图13所示, FlashDMoE 随 GPU 数量 线性扩展, 在 8 块 GPU 时达到 17.7 MTokens/s。这比 FasterMoE 高出 5.7 倍, 比 Megatron-TE 和 Megatron-CUTLASS 高出 4.9 倍。值得注意的是, 尽管 FlashDMoE 完全在 FP32 下运行, 而基线方法使用 FP16, 这些结果依然得以实现。这表明, FlashDMoE 的设计并非依赖低精度来消除吞吐量瓶颈, 而是通过最大化硬件利用率并消除主机驱动的低效来实现。

4.6 专家可扩展性

我们分析在固定序列长度 ($T = 16K$) 下, 随着专家数量增加, FlashDMoE 的缩放情况。请注意, 对于讨论的图表中, x 轴上的专家数量是跨所有 GPU 的总数。每个 GPU 分到该数值的 1/8。如图14所示, 即使专家数量从 8 增长到 128 名专家, FlashDMoE 仍能保持低且一致的延迟, 符合预期。相比之下,

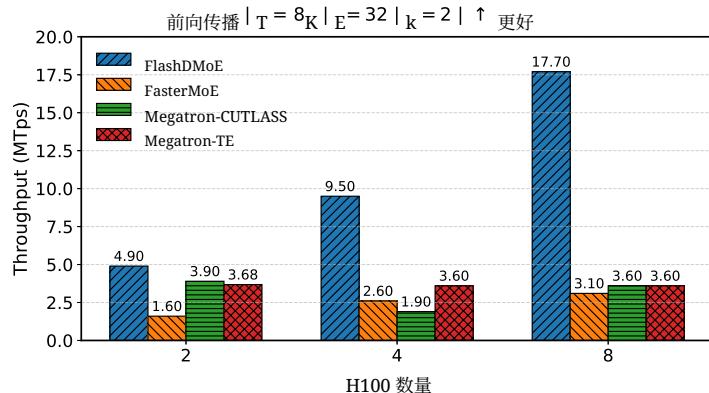


图13: GPU 数量增加时的吞吐量。我们将吞吐量计算为 $\frac{T * N_G}{latency}$, 其中 N_G 为 GPU 数量。

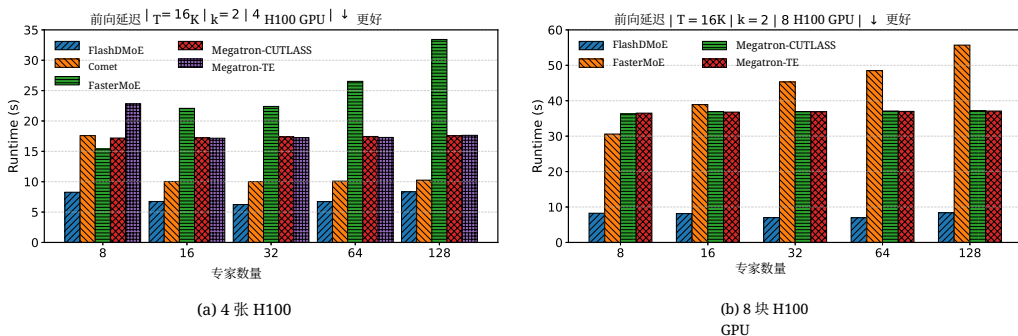


图14: *Number of experts*增加时的前向延迟。

由于内核启动开销增加，基线方法表现出超线性延迟增长。以 128 名专家为例，在 4 张 H100 上，FlashDMoE 相比这些基线方法可提升至多4倍；在 8 块 H100 GPU 上可提升6.6倍。FlashDMoE 的负载高效的通信与由调度器驱动的内核内分发，使其在不产生其他系统中所见的通信与编排开销的情况下，维持专家并行。以上结果进一步证明了 FlashDMoE 在超稀疏 MoE 配置下的可扩展性。

4.7 内存开销

我们测量了用于对称张量 L 以及 FlashDMoE 的运行时簿记状态所需的 GPU 内存。内存开销主要取决于分块大小、专家容量（EC）以及专家数量（ E ）。表3汇总了在各种配置下的内存开销，证实 FlashDMoE 保持着适度且可预测的内存占用。

5 局限性与未来工作

尽管 FlashDMoE 带来了性能提升和架构创新，但仍存在若干值得承认的局限性——既有实践层面也有概念层面——为未来研究打开了大门。

- **编程复杂度。** 开发完全融合、持久化内核是一项非平凡的工程任务。尽管 FlashDMoE 证明了此类内核的可行性和收益，但其构建需要对 GPU 架构、同步与分布式协议以及内存层次结构具有深厚的专业知识。这一高屏障限制了采用。未来工作可考虑采用编译器级抽象或 DSL（领域特定语言）来普及这一技术。
- **FP16 支持和共享内存访问模式。** 尽管现代 GPU 原生支持半精度计算，但将 FlashDMoE 适配到 FP16 对于

表3: FlashDMoE 的内存开销 (分块大小 $bM = 128$), $Size(T) = \text{Token} * 4KB$.

Token	专家	EC	最大值(bM, EC)	记录开销 (MB)	$Size(L)$ (MB)	总计 (MB)
4K	16	256	256	64.57	64.00	128.57
4K	32	128	128	64.55	64.00	128.55
4K	64	64	128	128.90	128.01	256.91
4K	128	32	128	257.96	256.02	513.98
8K	16	512	512	128.95	128.01	256.95
8K	32	256	256	128.90	128.01	256.91
8K	64	128	128	128.90	128.01	256.91
8K	128	64	128	258.15	256.02	514.17
16K	16	1024	1024	257.89	256.02	513.90
16K	32	512	512	257.79	256.02	513.81
16K	64	256	256	257.80	256.02	513.81
16K	128	128	128	258.53	256.02	514.54

处理器的计算算子。具体而言，我们手工调优的 **swizzle** 共享内存布局并不是 CUTLASS 的 **Collective Mainloop** 算子（我们用其来实现设备内 GEMM 运算）的最有效模板参数。这种次优配置会降低内存吞吐量，如第 H 节所示。要在 Ampere 架构 GPU 及以下平台上解决这一问题，需要对最优布局进行仔细研究；而在 Hopper 架构 GPU 及以上平台上，我们预计将在未来的改进中使用 CUTLASS 提供的构建器接口。

- **缺乏反向传播和训练支持。** 尽管本工作聚焦于推理，要支持训练，需要将反向计算和梯度通信融合进内核。为此需要对内存记账和任务描述符定义进行非平凡的修改。尽管如此，这仍是将该系统扩展为完全支持端到端训练的一个令人兴奋的方向。

6 结论

本文介绍了 FlashDMoE，这是首个将整个专家混合（MoE）算子融合进单个、持久化 GPU 内核的系统。我们表明，现有的专家混合实现存在两项关键低效：(1) 由 CPU 管理的同步通信，导致互连未被充分利用；(2) 通过多个 GPU 内核的碎片化执行，引入开销和同步延迟。

相较之下，FlashDMoE 采用 GPU 自主性模型，将计算、通信和调度嵌入到一个统一内核之中。它利用 Actor 风格并发、warp 专用化，以及异步 (R)DMA，实现细粒度的通信与计算重叠。

我们的评估显示，最高可达 **6× 加速比**，优于最先进的系统，GPU 利用率最高可达 **9×** 提升，以及 **5.7×** 分布式混合专家的吞吐量提升。FlashDMoE 挑战了分布式深度学习中的主导执行范式，并为构建未来的 GPU 原生系统提供了一个有说服力的模板。

尽管仍存在若干局限性，例如编程复杂度和缺乏 FP16 支持，这项作为内核内分布式计算的新时代奠定了基础。未来的系统可能在此基础上，为整个训练流水线实现内核融合，引领设计从 CPU 编排转向完全自主的 GPU 执行。

7 致谢

本研究得到美国国家科学基金会（NSF）奖项 #2444537 和 ACE 的资助，ACE 是 JUMP 2.0 的七个中心之一，JUMP 2.0 是由美国国防高级研究计划局（DARPA）赞助的半导体研究公司（SRC）项目。本工作还使用了国家能源研究科学计算中心（NERSC）的资源，该中心是美国能源部科学办公室用户设施，由美国能源部科学办公室根据合同编号 DE-AC02-05CH11231 提供支持，并使用 NERSC 奖项 ASCR-ERCAP0030076。我们感谢 Giulia Guidi 博士为我们提供对这些 NERSC 超级计算资源的访问。

参考文献

- [1] DeepSeek-AI. DeepSeek-V3 技术报告, 2025年。URL <https://arxiv.org/abs/2412.19437>.[2] Meta 人工智能。Llama 4 群: 原生多模态人工智能创新新时代的开端, 2025 年。URL <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.[3] Mosaic 研究。介绍 DBRX: 全新的最先进开源大型语言模型, 2024年。URL <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>.[4] Snowflake 人工智能研究。Snowflake Arctic: 面向企业级人工智能的最佳大型语言模型——高效智能, 真正开源, 2024年。URL <https://www.snowflake.com/en/blog/arctic-open-efficient-foundation-language-models-snowflake/>.[5] 阿希什·瓦斯瓦尼、诺姆·沙齐尔、妮基·帕尔马、雅各布·乌什科雷特、利昂·琼斯、艾丹·N·戈麦斯、卢卡什·凯泽, 以及伊利亚·波洛苏欣。注意力机制就是全部所需。见神经信息处理系统大会, 第30卷。Curran Associates 公司, 2017年。URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.[6] Siddharth Singh, Olatunji Ruwase, Ammar Ahmad Awan, Samyam Rajbhandari, Yuxiong He, and Abhinav Bhatele. 一种混合张量-专家-数据并行方法以优化专家混合训练。见第37届ACM国际超级计算会议论文集, ICS 2023, 页 203–214, 美国纽约州纽约市, 2023年。美国计算机协会。ISBN 9798400700569。数字对象唯一标识符: 10.1145/3577193.3593704。URL <https://doi.org/10.1145/3577193.3593704>.[7] J uncai Liu, Jessie Hui Wang, and Yimin Jiang. 雅努斯: 统一的分布式训练框架, 面向稀疏混合专家模型。见ACM SIGCOMM 2023会议论文集, ACM SIGCOMM 2023, 页 486–498, 美国纽约州纽约市, 2023年。美国计算机协会。ISBN 9798400702365。数字对象唯一标识符: 10.1145/3603269.3604869。URL <https://doi.org/10.1145/3603269.3604869>.[8] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, and Yida Wang. Lancet: 通过全图计算-通信重叠加速专家混合训练。见 P. Gibbons, G. Pekhimenko 和 C. De Sa (编者), 机器学习与系统会议论文集, 第6卷, 页 74–86, 2024年。URL https://proceedings.mlsys.org/paper_files/paper/2024/file/339caf45a6fa281cae8adc6465343464-Paper-Conference.pdf.[9] NVIDIA集体通信库 (NCCL)。<https://developer.nvidia.com/nccl>。
- [10] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley 和 Yuxiong He. DeepSpeed-MoE: 推进专家混合推理与训练以支撑下一代AI规模。载于第39届国际机器学习会议论文集, 收录于机器学习研究论文集第162卷, 页码 18332–18346。PMLR, 2022年7月17日至23日。URL <https://proceedings.mlr.press/v162/rajbhandari22a.html>。
- [11] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen 和 Xin Liu. Comet: 用于专家混合的细粒度计算-通信重叠。载于MLSys 2025。URL <https://arxiv.org/abs/2502.19811>。
- [12] 英伟达。Megatron-LM, 2025年。URL <https://github.com/NVIDIA/Megatron-LM?tab=readme-ov-file>。v0.11.0。
- [13] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, 和 Qin Li. FasterMoE: 大规模动态预训练模型的建模与训练优化。收录于第27届 ACM SIGPLAN 并行程序设计原理与实践研讨会论文集 (PPoPP 2022), 第 120–134 页, 2022年。

[14] Michael Wendt 和 Joshua Wyatt. CUDA 图入门。https://developer.nvidia.com/blog/cuda-graphs/, 2019年。访问日期: 2024-05-15。[15] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, 和 Matei Zaharia. 使用 Megatron-LM 在 GPU 集群上高效进行大规模语言模型训练。收录于国际高性能计算、网络、存储与分析大会论文集, SC '21, 美国纽约州纽约市, 2021年。美国计算机协会。国际标准书号 9781450384421。数字对象唯一标识符: 10.1145/3458817.3476209。URL https://doi.org/10.1145/3458817.3476209。[16] 英伟达。Nvidia OpenSHMEM 库 (NVSHMEM), 2025年。URL https://docs.nvidia.com/nvshmem/api/index.html。v3.2.5。[17] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, 和 Manish Gupta. CUTLASS, 2025年。URL https://github.com/NVIDIA/cutlass。[18] 国家能源研究科学计算中心 (NERSC)。网络 - NERSC 文档。https://docs.nersc.gov/performance/network/, 2025年。[访问日期 23-05-2025]。[19] C. Bell, D. Bonachea, R. Nishtala, 和 K. Yelick. 优化带宽受限问题: 使用单边通信与重叠。收录于第20届 IEEE 国际并行与分布式处理研讨会论文集, 页数 10 pp., 2006年。数字对象唯一标识符: 10.1109/IPDPS.2006.1639320。[20] Yuxin Chen, Benjamin Brock, Serban Porumbescu, Aydin Buluc, Katherine Yelick, 和 John Owens. Atos: 一种用于图分析的任务并行 GPU 调度器。收录于第51届 并行处理国际会议论文集, ICPP '22, 美国纽约州纽约市, 2023年。美国计算机协会。国际标准书号 9781450397339。数字对象唯一标识符: 10.1145/3545008.3545056。URL https://doi.org/10.1145/3545008.3545056。[21] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, HoYuen Chau, Peng Cheng, Fan Yang, Mao Yang, 和 Yongqiang Xiong. Tutel: 大规模自适应专家混合。由 D. Song、M. Carbin 和 T. Chen 主编, 收录于机器学习与系统会议论文集, 第 5 卷, 第 269–287 页。Curran, 2023年。URL https://proceedings.mlsys.org/paper_files/paper/2023/file/5616d34cf8ff73942cfd5aa922842556-Paper-mlsys2023.pdf。[22] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, 和 Qin Li. FasterMoE: 大规模动态预训练模型的建模与训练优化。收录于第27届 ACM SIGPLAN 并行程序设计原理与实践研讨会论文集, PPOPP '22, 第 120–134 页, 美国纽约州纽约市, 2022年。美国计算机协会。国际标准书号 9781450392044。数字对象唯一标识符: 10.1145/3503221.3508418。URL https://doi.org/10.1145/3503221.3508418。[23] Xiaonan Nie, Xupeng Miao, Zilong Wang, Zichao Yang, Jilong Xue, Lingxiao Ma, Gang Cao, 和 Bin Cui. FlexMoE: 通过动态设备放置缩放大规模稀疏预训练模型训练。ACM 数据管理论文集, 1(1), 2023年5月。数字对象唯一标识符: 10.1145/3588964。URL https://doi.org/10.1145/3588964。[24] Shaohuai Shi, Xinglin Pan, Qiang Wang, Chengjian Liu, Xiaozhe Ren, Zhongzhe Hu, Yu Yang, Bo Li, 和 Xiaowen Chu. ScheMoE: 一种可扩展的专家混合分布式训练系统, 具备任务调度功能。收录于第十九届欧洲计算机系统会议论文集, EuroSys '24, 第 236–249 页, 美国纽约州纽约市, 2024年。美国计算机协会。国际标准书号 9798400704376。数字对象唯一标识符: 10.1145/3627703.3650083。URL https://doi.org/10.1145/3627703.3650083。[25] Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, 和 Dazhao Cheng. 利用跨 GPU 共享内存, 实现无缝的 MoE 通信-计算融合。收录于第30届 ACM SIGPLAN 并行程序设计原理与实践年度研讨会论文集, PPOPP '25, 第 170–182 页, 美国纽约州纽约市, 2025年。美国计算机协会。国际标准书号 9798400714436。数字对象唯一标识符: 10.1145/3710848.3710868。URL https://doi.org/10.1145/3710848.3710868。

[26] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, 和 Christopher Ré. FlashAttention: 具有 I/O 感知的快速且内存高效的精确注意力。刊于神经信息处理系统进展, 第35卷, 第16344–16359页。Curran Associates 公司, 2022年。URL https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf. [27] Gul A. Agha. Actors: 分布式系统中并发计算的模型。技术报告, 1985年。麻省理工学院人工智能实验室技术报告。 [28] Carl Hewitt, Peter Bishop, 和 Richard Steiger。一种用于人工智能的通用模块化 Actor 形式体系。IJCAI'73, 第235–245页, 美国加利福尼亚州旧金山, 1973年。摩根·考夫曼出版社。 [29] Irene Greif。通信并行进程的语义。博士论文, 麻省理工学院, 1975年。 [30] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, 和 Xiaowen Chu。对 Nvidia Hopper GPU 架构的基准评测与剖析。刊于2024 IEEE国际并行与分布式处理研讨会 (IPDPS), 第656–667页, 美国加利福尼亚州洛杉矶阿拉米托斯, 2024年5月。IEEE计算机学会。数字对象唯一标识符: 10.1109/IPDPS57955.2024.00064。URL <https://doi.ieeecomputersociety.org/10.1109/IPDPS57955.2024.00064>. [31] Hamdy Abdelkhalik, Yehia Arafa, Nandakishore Santhi, 和 Abdel-Hameed Badawy。通过微基准测试与指令级分析揭示 NVIDIA Ampere 架构的奥秘, 2022年。URL <https://arxiv.org/abs/2208.11174>. [32] NVIDIA。PTX 指令集架构: 版本 8.7, 2025年。URL https://docs.nvidia.com/cuda/pdf/ptx_isa_8.7.pdf. [33] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, 和 Zhifeng Chen。GShard: 通过条件计算与自动分片扩展巨型模型。载于第九届国际学习表征会议, ICLR 2021, 线上会议, 奥地利, 2021年5月3–7日。OpenReview.net, 2021年。URL <https://openreview.net/forum?id=qrwe7XHTmYb>. [34] Katherine Yelick, Dan Bonachea, Wei-Yu Chen, Phillip Colella, Kaushik Datta, Jason Duell, Susan L. Graham, Paul Hargrove, Paul Hilfinger, Parry Husbands, Costin Iancu, Amir Kamil, Rajesh Nishtala, Jimmy Su, Michael Welcome, 和 Tong Wen。使用分区全局地址空间语言的生产力与性能。载于并行符号计算国际研讨会, PASC0 '07, 第24–32页, 美国纽约州纽约市, 2007年。美国计算机协会。国际标准书号 9781595937414。数字对象唯一标识符: 10.1145/1278177.1278183。URL <https://doi.org/10.1145/1278177.1278183>. [35] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, 和 Xin Liu。Triton-distributed: 使用 Triton 编译器在分布式AI系统上编程重叠内核, 2025年。URL <https://arxiv.org/abs/2504.19442>. [36] Quentin Anthony, Yury Tokpanov, Paolo Glorioso, 和 Beren Millidge。BlackMamba: 用于状态空间模型的专家混合, 2024年。URL <https://arxiv.org/abs/2402.01771>. [37] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostafa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, 等。使用 Megatron-LM 在 GPU 集群上高效进行大规模语言模型训练。载于国际高性能计算、网络、存储与分析大会论文集, 第1–15页, 2021年。 [38] NVIDIA。Transformer 引擎, 。URL <https://github.com/NVIDIA/TransformerEngine>. [39] 字节跳动。Flux 的重叠性能比非重叠更差 [4xrtx 4090], 2025。URL <https://github.com/bytedance/flux/issues/111#issuecomment-2822823236>.

[40] NVIDIA. NVIDIA Nsight Systems 指标, 。 URL <https://docs.nvidia.com/nsight-systems/UserGuide/index.html?highlight=SM%2520active#available-metrics>. [41] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, 和 Olli Saarikivi. 打破分布式机器学习工作负载中计算与通信的抽象屏障。载于第27届ACM编程语言与操作系统体系结构支持国际会议 (ASPLOS 22) 论文集, 第402–416页, 2022年。 [42] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, 等。通过分解在大型深度学习模型中将通信与依赖计算重叠。载于第28届ACM编程语言与操作系统体系结构支持国际会议 (ASPLOS 22) 论文集, 第93–106页, 2022年。 [43] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, 和 Chao Yang. Centauri: 通过通信划分在大模型训练中实现高效的通信-计算重叠调度。载于第29届ACM编程语言与操作系统体系结构支持国际会议 (ASPLOS 24) 论文集, 第178–191页, 2024年。 [44] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, 和 Matthew D Sinclair. T3: 用于计算与集体通信细粒度重叠的透明跟踪与触发。载于第29届ACM编程语言与操作系统体系结构支持国际会议 (ASPLOS 24) 论文集, 第1146–1164页, 2024年。 [45] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, 等。{MegaScale}: 将大语言模型训练扩展到超过 10,000 {GPU}。载于第21届USENIX 网络系统设计与实现研讨会 (NSDI 24) , 第745–760页, 2024年。 [46] Weigao Sun, Zhen Qin, Weixuan Sun, Shidi Li, Dong Li, Xuyang Shen, Yu Qiao, 和 Yiran Zhong. CO2: 通过完全通信-计算重叠实现高效分布式训练。载于第十二届国际学习表征会议 (ICLR 24) , 2024年。 [47] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, 和 Aditya Akella. 更好地协同: 使用 {SYNDICATE} 联合优化 {ML} 集体通信调度与执行规划。载于第20届USENIX 网络系统设计与实现研讨会 (NSDI 23) , 第809–824页, 2023年。 [48] Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, 和 Dazhao Cheng. 利用跨 GPU 共享内存实现无缝的 MoE 通信-计算融合。载于第30届ACM SIGPLAN 并行程序设计原理与实践年度研讨会论文集, 第170–182页, 2025年。 [49] Kishore Punniyamurthy, Khaled Hamidouche, 和 Bradford M Beckmann. 通过融合的计算-集体通信操作优化分布式 ML 通信。载于SC24: 国际高性能计算、网络、存储与分析大会, 第1–17页, 2024年。 [50] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, 等。Tutel: 可扩展的自适应专家混合。机器学习与系统会议论文集 (MLSys 23) , 5:269–287, 2023年。 [51] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, 和 Yida Wang. Lancet: 通过全图计算-通信重叠加速专家混合训练。载于机器学习与系统会议论文集 (MLSys 24) , 第74–86页, 2024年。 [52] OpenFabrics 接口工作组. fi_cxi. https://ofiwg.github.io/libfabric/v1.21.0/man/fi_cxi.7.html, 2025年。 [访问日期 23-05-2025]。

NeurIPS 论文检查清单

1. 主张

问题：摘要与引言中提出的主要主张是否准确反映了论文的贡献和范围？

答案：[是]

2. 局限性

问题：论文是否讨论了作者所进行工作的局限性？

答案：[是]

3. 理论假设与证明

问题：对于每个理论结果，论文是否提供了完整的假设集合以及一份完成（且正确）的证明？

答案：[NA]

4. 实验结果可复现性

问题：该论文是否完全披露了复现该论文主要实验结果所需的全部信息，以至于影响该论文的主要主张和/或结论（无论是否提供代码和数据）？

回答：[是]

5. 对数据和代码的开放获取

问题：该论文是否提供对数据和代码的开放获取，并附有足够的说明，以便按照补充材料中的描述，忠实复现主要实验结果？

回答：[是]

6. 实验设置/细节

问题：该论文是否详细说明了理解结果所需的全部训练和测试细节（例如，数据划分、超参数、它们的选择方式、优化器类型等）？

回答：[不适用]

7. 实验的统计显著性

问题：该论文是否恰当地报告并正确定义了误差条，或提供了关于实验的统计显著性的其他适当信息？

回答：[是]

理由：评估部分中报告的所有结果都是通过150次运行的平均值得到的。

8. 实验计算资源

问题：对于每个实验，论文是否提供了足够的信息，说明复现实验所需的计算机资源（计算工作节点类型、内存、执行时间）？

回答：[是]

9. 伦理守则

问题：论文开展的研究在各方面是否符合 NeurIPS 伦理守则

<https://neurips.cc/public/EthicsGuidelines>?

回答：[是]

10. 更广泛的影响

问题：该论文是否同时讨论了所开展工作的潜在的积极社会影响和负面社会影响？

回答：[不适用]

理由：我们不预见会有短期内的社会或伦理影响，但我们承认，提升的计算效率可能会放大获取大规模模型的能力，这引发了对一些普遍问题的总体考量，例如训练的环境成本以及负责任的下游使用。我们建议本系统的用户在将其集成进更广泛的机器学习应用时，考虑这些因素。

11. 保障措施

问题：论文是否描述了为负责任地发布具有高滥用风险的数据或模型而已实施的保障措施（例如，预训练语言模型、图像生成器或爬取的数据集）？

答案： [NA]

12. 现有资产的许可证

问题：是否对论文中使用的资产（例如，代码、数据、模型）的创作者或原始所有者进行了适当署名，并且是否明确说明并恰当遵守了许可证和使用条款？

答案： [Yes]

13. 新资产

问题：论文中引入的新资产是否有完善的文档，并且文档是否随资产一同提供？

回答： [是]

14. 众包与涉及人类受试者的研究

问题：对于众包实验和涉及人类受试者的研究，论文是否包含提供给参与者的完整的说明文本及（如适用）截图，以及关于报酬（如果有）的详细信息？

回答： [不适用]

15. 机构审查委员会（IRB）对涉及人类受试者的研究的批准或同等审批

问题：该论文是否描述了研究参与者可能承担的潜在风险、这些风险是否已向受试者披露，以及是否获得了机构审查委员会（IRB）批准（或基于您所在国家或机构要求的同等审批/审查）？

回答： [NA]

16. 大型语言模型使用声明

问题：如果 LLMs 是本项研究核心方法中的重要、具有原创性或非标准组成部分，论文是否描述了其使用情况？请注意：如果大型语言模型仅用于写作、编辑或排版用途，且不影响研究的核心方法论、科学严谨性或原创性，则无需进行声明。

答案： [NA]

A 相关工作

计算-通信重叠与内核融合。 为降低分布式DNN训练中同步带来的通信开销，许多研究致力于提高计算与通信的重叠度。对于不含MoE层的通用基于Transformer的模型，许多工作 [41–49] 提出了将计算与通信操作进行划分和调度的见解与技术，旨在实现更细粒度的重叠。为应对MoE训练中 $AllToAll$ （全互连）通信与专家并行带来的挑战，Tutel [50]和 FasterMoE [13] 将 $AllToAll$ （全互连）与专家计算进行重叠。Lancet [51] 进一步使前向传播中的非MoE计算以及反向传播中的权重梯度计算都能够与 $AllToAll$ （全互连）重叠。尽管实现了重叠，这些方法在实际中仍因带有屏障的阻塞式同步集体通信而性能受限。相比之下，FlashD-MoE 通过将异步、设备发起的数据传输与分块计算重叠，从根本上消除了这些低效，且全部在单一内核内。FlashDMoE 进一步区别于诸如 COMET [11] 和 DeepEP [1]，等同样采用这种内核发起的通信形式的SOTA工作，后者仅具有粗粒度，且缺乏完整的内核融合。

B 动机图表

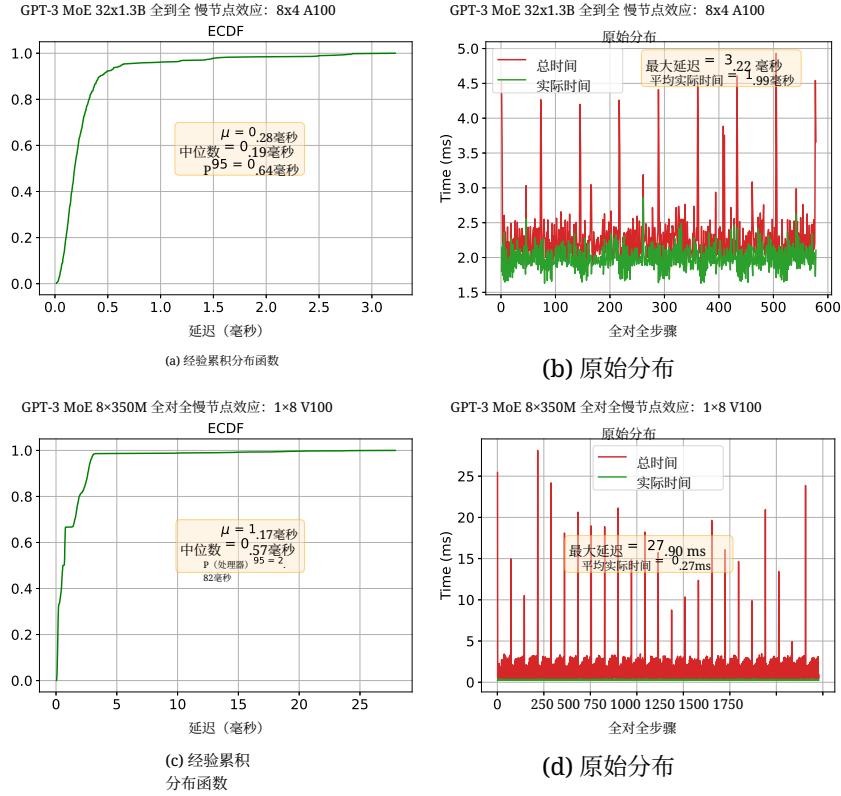


图15: 同步 $AllToAll$ （全互连）的掉队者效应。 $M \times N$ A100 或 V100 表示单个节点内的 N 个 GPU，横跨 M 个节点。每个 GPU 在每个 $AllToAll$ （全互连）步骤中都会与所有其他 GPU 通信。我们捕获了跨多个步骤由掉队者引起的延迟分布。**实际时间** t_a 表示所有 GPU 中最快的内核执行时间；相反，**总时间** t 是最大记录的 step 时间，而 $Delay$ 是 t 与 t_a 之间的最大差值。注意 $Delay$ 是空闲时间。

C 定理3.1 的证明

We b以两个必要且至关重要的定义开始

证明。

定义 C.1. 将写操作定义为 $w(p_s p_b i)$, 其中 p_s 为源进程, i 为有序元组, 用于指示驻留在目标进程 p_t 上的 L 的索引坐标。写-写冲突发生于至少存在两个不同的、未同步的并发写操作 $w_1(p_{s_1} p_{t_1} i_1)$ 并且 $w_2(p_{s_2} p_{t_2} i_2)$, 使得 $p_{t_1} = p_{t_2}$ 并且索引坐标 $i_1 = i_2$ 但 $p_{s_1} \neq p_{s_2}$

定义 C.2. 对于任意源进程 p_s , 一个有效索引坐标 $i = (p^*, r, b, e, c)$ 满足以下条件:

1. 对于跨设备写操作, 必须满足 $p^* = p_s$ 和 $b = 1$ 。注意, 这也适用于自环写操作 $w(p_s p_b i)$ 。
2. 对于任意写操作 $w(p_s p_b i)$, 若 $b = 0$, 则 $p_s = p_t$ 。该规则描述了设备内暂存写操作。

我们在下文重述定理3.1并概述其证明。

定理 C.1. 对称张量布局 L 是写写无冲突的。

证明。与典型的物理实现相同, 假设每个索引坐标 i 在 L 中映射到一个唯一的内存段。接下来, 我们用反证法证明: 在使用有效 i 访问 L 时, 不会存在写-写冲突。为简单起见, 在描述写操作时我们仅包含索引坐标。假设至少存在两个写操作 $w_1(p_{s_1} p_{t_1} i_1)$ $w_2(p_{s_2} p_{t_2} i_2)$, 其具有 $p_{t_1} = p_{t_2}$ 且具有有效的目标坐标 i_1, i_2 , 其中 $i_1 = i_2$ 按字典序, 二者在下文展开。

$$i_1 = (p_1, r_1, b_1, e_1, c_1), i_2 = (p_2, r_2, b_2, e_2, c_2)$$

请注意, 对于消息暂存状态, 尽管 $i_1 = i_2$, 最终的内存段分别驻留在 p_{s_1} 和 p_{s_2} 中的不同物理缓冲区中。因此, 在该状态下, 不会产生冲突, 因为进程内写操作始终具有不同的 c_j 坐标, 其中 $j \in \{0, C-1\}$ 。对于跨进程传输, 我们有两种情形。

情形 1: $p_{s_1} = p_{s_2}$

此处, w_1 和 w_2 是相同的操作。这与冲突的定义相矛盾, 该定义要求 $p_{s_1} \neq p_{s_2}$ 。在实践中, 此类重复写操作甚至不会发生。

情形 2: $p_{s_1} \neq p_{s_2}$

为确保 i_1 和 i_2 的有效性, 需要满足 $p_1 = p_{s_1}$ 和 $p_2 = p_{s_2}$ 。然而, 这意味着 $i_1 \neq i_2$, 从而得到所需的矛盾。□

D (FFN 中间维度) 任务实现

```

1 宏定义GEMM运算2 2
2 结构体t_align(16)任务Task {
3     常量限定符字节指针a数据;
4     数组a常量限定符字节指针, GEMM运算b数据;
5     数组a字节指针*, GEMM运算c数据;
6     数组a常量限定符字节指针, GEMM运算d数据;
7     字节指针e数据;
8     无符号整数 64_t flags;
9     无符号整数M;
10    无符号整数同步系数;
11    无符号整数块索引;
12    无符号整数批次索引;
13    无符号整数对等系数;
14    无符号整数expert Idx;
15    无符号整数isPer Remote;
16    TaskType taskType;
17    无符号整数 16_t tileSize;
18    //填充至128字节缓存行e cache line
19    无符号整数padding[6] = {0};
20 }

```

图16: 任务结构体。任务类型 $\in \{GEMM_0, GEMM_1, Combine\}$

E 参与者

E.1 处理器

算法2: 处理器参与者: 由一个块执行

```
1 开始
2    $tQ \leftarrow \text{获取任务队列}()$ 
3    $signal \leftarrow 0$ 
4   // 共享内存变量
5    $task \leftarrow \{\}$ 
6    $interrupt \leftarrow \text{False}$ 
7    $complete \leftarrow \text{False}$ 
8   while  $interrupt == \text{False}$  do
9       if  $warpId == 0$  then
10           if  $threadId == 0$  then
11               等待来自调度器的任务( $interrupt, signal$ )
12               FencedNotifyRQ( $ready$ )
13           结束 if
14           syncwarp()
15           warpReadTQ( $tQ, signal, task$ )
16       结束 if
17       syncthreads()
18       if  $interrupt == \text{False}$  then
19           switch 任务类型 do
20               case  $GEMM_0$  do
21                   // 融合的通用矩阵乘法、后处理和异步瓦片预取
22                   fGET( $GEMM_0, task$ )
23                   if  $threadId == 0$  then
24                        $complete \leftarrow \text{NotifyTileCompletion}()$ 
25                   结束 if
26                   syncthreads()
27                   if  $complete == \text{真}$  then
28                       NotifySchedulerNextGEMM( $tQ$ )
29                   结束 if
30               end case
31               case  $GEMM_1$  do
32                   // 融合的通用矩阵乘法、后处理和异步分块传输
33                   fGET( $GEMM_1, task$ )
34               end case
35               case  $Combined$  do
36                   合并( $task$ )
37               结束 case
38           结束 switch
39       结束 if
40   结束 while
41 结束
```

E.2 调度器

算法 3: 调度器 *Actor*: 由一个线程束执行

```
1 开始
2    $scheduled \leftarrow 0$ 
3    $tTB \leftarrow 0$ 
4    $tqState \leftarrow \{\}$ 
5    $pTDB \leftarrow \text{GetProcessorDoorbell}()$ 
6    $sTDB \leftarrow \text{Get订阅者Doorbell}()$ 
7    $taskBound \leftarrow \text{GetTaskBound}()$ 
8    $tTB \leftarrow \text{原子加载}(taskBound)$ 
9   // 环形缓冲区就绪队列
10   $rQ \leftarrow \{\}$ 
11  // 使用处理器ID填充就绪队列
12  填充就绪队列( $rQ$ )
13  while  $scheduled < tTB$  do
14       $lt \leftarrow 0$ 
15      并行执行
16          | 扫描门铃, 并将观测到的任务计数填充到  $tqState$ 
17          | 将本地观测到的任务计数汇总到  $lt$ 
18      end
19       $qS, taskTally \leftarrow 0$ 
20      //  $qS$  是包含式输出
21      Warp 包含式求和( $lt, qS, tasktally$ )
22      while  $tasktally > 0$  do
23          | 重新填充  $rQ$ 为就绪处理器ID
24          | 并行执行
25          |     | 从  $rQ[qS]$ 开始, 向处理器发送关于来自  $tqState$ 的任务索引的信号
26          |     end
27      结束 while
28      如果  $threadId == 0$ 则
29          |  $tTB \leftarrow \text{原子加载}(taskBound)$ 
30      结束 if
31       $tTB \leftarrow \text{Warp广播}(tTB)$ 
32  结束 while
33  中断订阅者()
34  中断处理器()
35 结束
```

E.3 订阅者

算法 4: 订阅者执行体: 由三个warp (束) 执行

输入: $T_\phi \in (\mathbb{R}^2)^{E \times C}$, $G_\phi \in \mathbb{R}^{S \times E}$ $O \in \mathbb{R}^{S \times H}$, $X \in \mathbb{R}^{E \times H \times D}$

```

1 开始
2   $interrupt \leftarrow \text{GetSharedInterrupt}()$ 
3   $flags \leftarrow \text{GetSymmetricFlags}()$ 
4   $tQ \leftarrow \text{获取任务队列}()$ 
5  // 任务数量的预定义上界。
6  // 我们将该数值调整为计算得到的实际任务计数
7  // 该计数由从对等 GPU 接收到的分发信号计算得出
8   $taskBound \leftarrow \text{GetTaskBound}()$ 
9  while 原子加载( $interrupt$ ) == False do
10     // 分发标志
11     并行执行
12         访问分发标志
13         以原子方式获取信号
14         if 信号已设置且标志未访问 then
15             标记为已访问
16             自校正任务边界( $taskBound, Signal$ )
17             在消费数据包之前强制执行内存一致性
18             使用  $X$  将数据包解码为一组  $GEMM_0$  任务描述符
19             对任务描述符执行写操作至  $tQ$ 
20             通知调度器解码后的任务
21         结束 if
22     end
23     将标志前移调度标志位长度
24     以原子方式获取信号
25     // 合并信号
26     并行执行
27         访问合并标志: 每个瓦片一个
28         如果 信号已设置且标志未访问 则
29             标记为已访问
30             在处理数据包之前强制内存一致性
31             将数据包解码为一组  $combine$  任务描述符 使用  $T_\phi, G_\phi, O$ 
32             对任务描述符执行写操作到  $tQ$ 
33             通知调度器解码后的任务
34         结束 if
35     end
36 结束 while
三十七 结束

```

F 多节点评估

F.1 设置

在本实验中，我们希望在多节点设置下评估 FlashDMoE。我们使用 4 个节点，每个节点由 4 块 A100 GPU 组成，彼此通过 NVLink 完全互连。跨节点时，每个 GPU 使用一块 NIC，提供 25 GB/s 的带宽。我们将专家数量设为 16，并让每个 GPU 仅承载一个，因此本地专家数为 1。注意，我们正式将最大汇聚数据量定义如下：

$$MIV = \frac{Tokens}{Experts} * local_experts * precision * hidden_size * 2 * n_{rg}$$

其中， n_{rg} 为远程对等体的数量，乘以 2 的因子用于计入通信轮次（分发与合并）。 $n_{rg} = 12$ 用于本实验。

F.2 结果

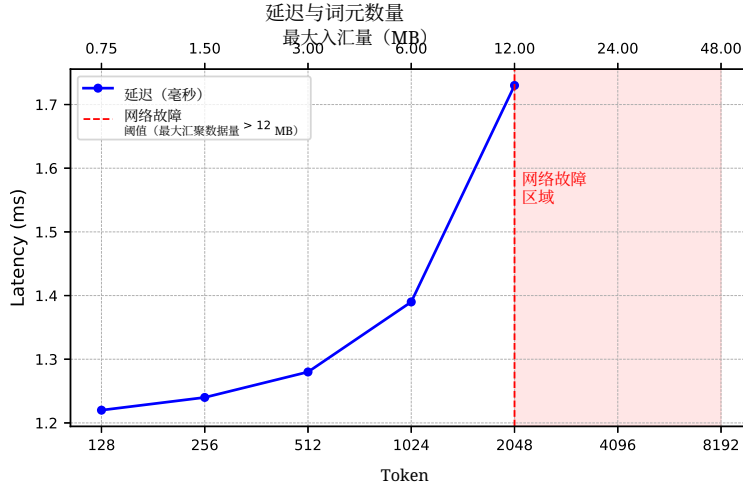


图17：多节点延迟评估。嵌入维度为 1024，FFN中间层维度为 4096。我们将最大汇聚数据量（MIV）定义为在单次汇聚拥塞事件中，网络接口卡接收的数据量的最坏情况上界。

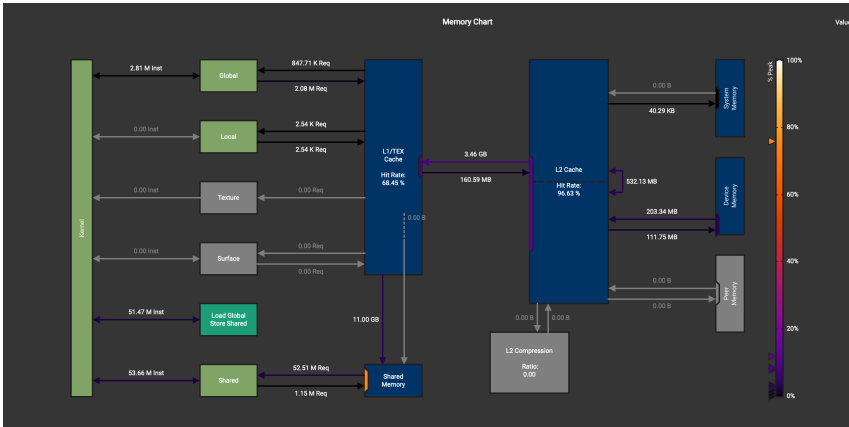
随着我们缩放令牌数量，延迟呈现次线性增长。然而，我们在 $Tokens > 2048$ 观察到，由于未能接收到预期消息，应用程序无法终止。我们推测，失败的原因是网络硬件层发生了缓冲区溢出，这在会生成许多且大型的消息的应用中很常见 [18] 例如我们的系统。我们注意到，通过调优硬件配置 [52] 可以解决该问题，但我们认为这方面的探索与本文工作不直接相关。

G 实现

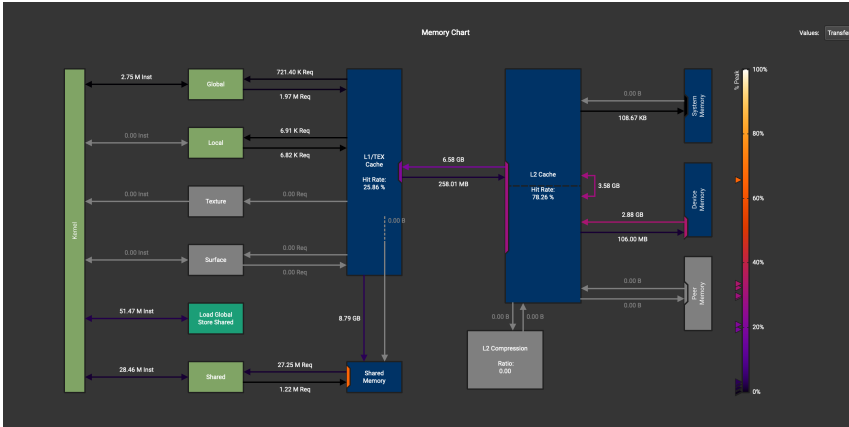
表4: FlashDMoE 的实现指标。

指标	数值
代码总行数 (CUDA/C++)	6820
内核栈帧大小	0 B
溢出写 (每线程)	0
溢出读 (每线程)	0
共享内存使用量 (每个线程块)	46 KB
每线程寄存器数	255
每个SM的最大活动线程块数	2
编译时间	53 秒
二进制大小	29 兆字节

H（嵌入维度） FP16内存吞吐量



(a) FP16 的内存子系统吞吐量



(b) FP32 的内存子系统吞吐量

图18: 在此，我们报告了 FlashDMoE 的 FP16（上）和 FP32（下）两种变体在 A100 上的内存子系统吞吐量总计。值得注意的是，在相同的工作负载下，FP16 实现相比其 FP32 对应版本，发出的共享内存指令数量多约 2 $\times$ 。我们将这种低效归因于 FlashDMoE 在处理半精度数据时采用的次优共享内存布局。尽管这一瓶颈可以通过改进的布局策略加以解决，但由于时间限制，我们将其留待未来工作。